

SBD.cpp : Readme

The code SBD_S2(DS).cpp is code used in conjunction with the SOLO float family, cpp json parsers. It's purpose is to process the Iridium SBD time/positions that are included within the hex files. These data are not processed by the L0 parser.

It should require no additional environmental setup beyond what is required for the L0 parser.

`g++ -g -std=c++20 src/L1/SBD_S2(DS).cpp -o SBD`

The source code resides in /src/L1 alongside the L0toL1_S2(DS).cpp code.

It requires the previous creation of the L0 json. Calling the routine requires 2 parameters, float sn and cycle number. : `"/SBD 3334 -1"`

What does SBD.cpp accomplish?:

- 1) It reads the indicated L0 json for the start/end of that cycles surface interval. It queries a series of cycle data until a valid date is found (first GPS_END, second Rise, third Fall, final GPS_START). The end of the surface interval is calculated as "start + ARGO_Mission["target_surface_time"].
- 2) Collects the SBD Iridium positions from the hex files that fall between the surface interval start/end.
 - a) The hex files contain data sorted by cycle, NOT by the SBD transmissions made during that cycle.
 - b) For a given cycle X, the transmissions made during that cycle could be collected from any cycle < X. Thus the code needs to search many hex files and the run time will increase in higher cycles of a float. To limit the run time, the code checks the "last modification time" of the hex file. If the file times tamp is prior to the surface time of cycle X the hex file is ignored. Thus knowledge of the "last modification time" of the hex files on your system are required to make sure this code is properly accessing the necessary hex files. I've tested this on linux. I attempted to include code from a windows operating system...but it is untested.
- 3) Outputs the SBD Iridium position data to a new json cycle file.
`"/data/SN/json/SBD/AOMLid_TRANSid_SBD_CYN.json".`
An example name is "10085_003333_SBD_008.json". If the SBD directory does not exist, the code creates it. Details of the contents are below.
- 4) Calculates the weighted average of the Iridium positions as described in the Argo manuals.

The final goal of SBD_S2(DS).cpp

- 1) weighted Iridium position if there is no GPS provided in the cycle.
- 2) Improved estimate of MC700, MC702, MC704 and MC800.

The SBD json: Here is an example of the output from SBD_S2(DS).cpp: 9044_003104_SBD_000.json

```
"sbd_iridium_position_summary": [  
  { "source": "SBD", "TIME": "2022-11-18T09:22:05Z", "JULD": 26619.39034, "momsn": 8,  
    "LATITUDE_iridium": -42.00734, "LONGITUDE_iridium": 180.00000, "error_iridium": 3000,  
    "sbd_contains_cycle": 0 },  
  { "source": "SBD", "TIME": "2022-11-18T09:22:18Z", "JULD": 26619.39049, "momsn": 9,  
    "LATITUDE_iridium": -42.00734, "LONGITUDE_iridium": 180.00000, "error_iridium": 3000,  
    "sbd_contains_cycle": 0 }  
],  
"sbd_weighted_iridium_position": [  
  { "JULD": 26619.39041, "LATITUDE": -42.007, "LONGITUDE": 180,  
    "POSITION_ERROR_ESTIMATED": 3000,  
    "POSITION_ERROR_COMMENT": "A subset of the available Iridium Positions were used that  
are deemed 'independent': 1; Error was computed as the square root of the inverse sum of the weights,  
where each weight is the inverse square of the individual measurement error: Weights=1/(error^2)" }  
]
```

This cycle 0 sends limited data in 2 SBD message packets. Both are listed in "sbd_iridium_position_summary". Please keep in mind that the json "telemetry summary" and "sbd_iridium_position_summary" will not always contain the same grouping of messages, even though in most files they will contain the same grouping.

In "sbd_iridium_position_summary" I've included the following data items as transmitted by the Iridium system.

"source" : This will always be "SBD" as there is no Iridium position with rudics

"TIME" : This is the satellite time stamp of the SBD transmission (Important: This is NOT the float clock time stamp)

"momsn" : The running count of transmissions of this float

"LATITUDE_iridium" : The SBD Iridium position calculated by the Iridium system

"LONGITUDE_iridium" : The SBD Iridium position calculated by the Iridium system

"error_iridium" : The estimated error (m) of the position calculated by Iridium

In addition I've included 2 informational add-ons.

"sbd_contains_cycle" : This contains the cycle number of the data included within the SBD message.

Typically, this value will be the same as the cycle of the float. However, if this particular SBE message was transmitted in a later cycle, it will be clearly shown here.

"JULD" : The Argo defined JULD (days since Jan 1, 1950) of the "TIME" date/time.

The "sbd_weighted_iridium_position" is a "value-added" data. It contains fields that align with eventual use in the Argo netCDF data files.

"JULD" : The Argo defined "days since Jan 1, 1950" averaged across all JULD records from

"sbd_iridium_position_summary"

"LATITUDE" : The weighted average of "independent" SBD latitude estimates.

"LONGITUDE" : The weighted average of "independent" SBD latitude estimates.

"POSITION_ERROR_ESTIMATED" : The error of the weighted average ("independent") errors.

"POSITION_ERROR_COMMENT" : A general, hardwired comment that explains the error assigned.

These 5 variables are named for exact netCDF variables found within the profile netCDF.

After writing the SBD_S2(DS).cpp code, I made 2 modifications to the L0toL1_S2(DS).cpp code.

1) If the SBD.json is available in (./data/SN/json/SBD), it is added to the L1 json. If I was to be strict, I would only add "sbd_iridium_position_summary". "sbd_weighted_iridium_position" are value-added fields and so should be added to a future file. However, since I was doing it I decided to add them both.

2) In writing this code I had to develop a subroutine to calculate "days since Jan 1, 1950". For future use, I've added this routine to L0toL1_S2(DS).cpp and added the "JULD" field to all "TIME" fields within the json ("Rise", "Fall", some "Pump" firmwares, "telemetry_summary", "GPS")